

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_org = pd.read_excel("/content/drive/MyDrive/capstone/datasets/0_original.xlsx")
df_org.head()
```

	data_source	full_name	age	age_group	family_members	position_in_siblings	gender	maritual_status	duration_of_separataion	profession_group
0	https://www.dhakatribune.com/bangladesh/nation...	Parvez	NaN	Teen	NaN	NaN	Male	NaN	NaN	Worke
1	https://www.dhakatribune.com/bangladesh/nation...	Alhaj Uddin	20.0	Teen	NaN	NaN	Male	relationship-rejected	NaN	NaN
2	https://www.dhakatribune.com/bangladesh/nation...	Priti Akter	14.0	Teen	NaN	NaN	Female	unmarried	NaN	Studen
3	https://www.dhakatribune.com/bangladesh/nation...	Acchia Akhter	NaN	NaN	NaN	NaN	Female	relationship-rejected	NaN	Studen
4	https://www.dhakatribune.com/bangladesh/dhaka/...	Dilshad Nahar	18.0	Teen	NaN	NaN	Female	relationship	NaN	Studen

5 rows × 36 columns

```
df_org.columns
```

```
Index(['data_source', 'full_name', 'age', 'age_group', 'family_members',
      'position_in_siblings', 'gender', 'maritual_status',
      'duration_of_separataion', 'profession_group', 'profession_description',
      'workplace', 'religion', 'financial_condition', 'hometown', 'latitude',
      'longitude', 'reason', 'reason_description', 'time', 'method',
      'previous_health_condition', 'previous_suicide_attempt', 'suicide_date',
      'unix_time', 'feels_like', 'temp_min', 'temp_max', 'air_pressure',
```

```
'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
'weather_description', 'Unnamed: 35'],
dtype='object')
```

```
# dropping unnecessary columns
df_org = df_org.drop(columns=['data_source', 'full_name'])
df_org.head()
```

	age	age_group	family_members	position_in_siblings	gender	marital_status	duration_of_separataion	profession_group	profession_description	workplace	...	temp_min
0	NaN	Teen	NaN	NaN	Male	NaN	NaN	Worker	he was an plumber	village	...	299.15
1	20.0	Teen	NaN	NaN	Male	relationship-rejected	NaN	NaN	NaN	NaN	...	295.57
2	14.0	Teen	NaN	NaN	Female	unmarried	NaN	Student	student of class nine	urban	...	301.15
3	NaN	NaN	NaN	NaN	Female	relationship-rejected	NaN	Student	student of Bangla Department	urban	...	299.34
4	18.0	Teen	NaN	NaN	Female	relationship	NaN	Student	studied fashion design at a private university	urban	...	300.15

5 rows × 34 columns

```
df_org.shape
```

```
(1617, 34)
```

```
# Calculate the percentage of missing data for each column
missing_percentage = df_org.isna().mean() * 100
```

```
# Create a DataFrame for the missing data summary
missing_data_summary = pd.DataFrame({
    'Column': missing_percentage.index,
    'Missing Percentage': missing_percentage.values
})
```

```
# Define the four groups based on missing percentage
group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage']]
group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage']]
group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage']]
```

```
# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
```

```
# Rename columns for clarity
group_1.columns = ['Column', 'Missing %']
group_2.columns = ['Column', 'Missing %']
group_3.columns = ['Column', 'Missing %']
group_4.columns = ['Column', 'Missing %']
```

```
# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))
```

Group 1: Columns with >70% missing data

Column	Missing %
Unnamed: 35	99.938157
duration_of_separataion	96.165739
position_in_siblings	93.135436
previous_suicide_attempt	91.651206
previous_health_condition	91.280148
family_members	77.612863

Group 2: Columns with >40% to ≤70% missing data

Column	Missing %
temp_min	52.999382
feels_like	52.999382
longitude	52.999382
wind_speed	52.999382
air_humidity	52.999382
air_pressure	52.999382
temp_max	52.999382
wind_deg	52.999382
clouds_sky	52.999382
weather_main	52.999382
weather_description	52.999382
latitude	52.937539
unix_time	52.380952
profession_description	49.412492
financial_condition	45.454545

Group 3: Columns with >20% to ≤40% missing data

Column	Missing %
workplace	37.847866
maritual_status	23.933210

Group 4: Columns with ≤20% missing data

Column	Missing %
reason_description	16.635745
profession_group	14.780458
age	12.739641
reason	12.306741
time	11.440940
age_group	5.627706
method	2.906617
religion	2.411874
suicide_date	2.040816
hometown	1.669759
gender	0.432900

▼ clearing data

```
df_clr = df_org.loc[:, missing_percentage <= 70].copy()
df_clr.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1617 entries, 0 to 1616
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   1411 non-null   float64
1   age_group             1526 non-null   object
2   gender                1610 non-null   object
3   marital_status        1230 non-null   object
4   profession_group      1378 non-null   object
5   profession_description 818 non-null    object
6   workplace             1005 non-null   object
7   religion              1578 non-null   object
8   financial_condition   882 non-null    object
9   hometown              1590 non-null   object
10  latitude              761 non-null    object
11  longitude             760 non-null    float64
12  reason                1418 non-null   object
13  reason_description    1348 non-null   object
14  time                  1432 non-null   object
15  method                1570 non-null   object
16  suicide_date          1584 non-null   object
17  unix_time             770 non-null    float64
18  feels_like            760 non-null    object
19  temp_min              760 non-null    object
20  temp_max              760 non-null    object
21  air_pressure          760 non-null    object
22  air_humidity          760 non-null    object
23  wind_speed            760 non-null    float64
24  wind_deg              760 non-null    object
25  clouds_sky            760 non-null    object
26  weather_main          760 non-null    object
27  weather_description    760 non-null    object
dtypes: float64(4), object(24)
memory usage: 353.8+ KB
```

```
df_clr = df_clr.rename(columns={'marital_status': 'marital_status'})
df_clr.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
      'profession_description', 'workplace', 'religion',
      'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
      'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
      'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
      'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
      'weather_description'],
      dtype='object')
```

✦ column lists

```
categorical_columns = [
    'age_group', 'gender', 'religion', 'marital_status', 'profession_group', 'workplace',
    'financial_condition', 'reason', 'time', 'method', 'weather_main', 'weather_description', 'hometown',
]
```

```
descriptive_columns = ['profession_description', 'reason_description']

numerical_columns = [
    'age', 'latitude', 'longitude', 'unix_time', 'feels_like', 'temp_min',
    'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky'
]

# only date format in ths database is "suicide_date"
```

```
for col in categorical_columns:
    print(f"Column '{col}': {df_clr[col].dtype}")
```

```
Column 'age_group': object
Column 'gender': object
Column 'religion': object
Column 'marital_status': object
Column 'profession_group': object
Column 'workplace': object
Column 'financial_condition': object
Column 'reason': object
Column 'time': object
Column 'method': object
Column 'weather_main': object
Column 'weather_description': object
Column 'hometown': object
```

```
for col in descriptive_columns:
    print(f"Column '{col}': {df_clr[col].dtype}")
```

```
Column 'profession_description': object
Column 'reason_description': object
```

```
for col in numerical_columns:
    print(f"Column '{col}': {df_clr[col].dtype}")
```

```
Column 'age': float64
Column 'latitude': object
Column 'longitude': float64
Column 'unix_time': float64
Column 'feels_like': object
Column 'temp_min': object
Column 'temp_max': object
Column 'air_pressure': object
Column 'air_humidity': object
Column 'wind_speed': float64
Column 'wind_deg': object
Column 'clouds_sky': object
```

```
import pandas as pd
for col in numerical_columns:
    df_clr[col] = pd.to_numeric(df_clr[col], errors='coerce')
```

```
for col in numerical_columns:
    print(f"Column '{col}': {df_clr[col].dtype}")
```

```

Column 'age': float64
Column 'latitude': float64
Column 'longitude': float64
Column 'unix_time': float64
Column 'feels_like': float64
Column 'temp_min': float64
Column 'temp_max': float64
Column 'air_pressure': float64
Column 'air_humidity': float64
Column 'wind_speed': float64
Column 'wind_deg': float64
Column 'clouds_sky': float64

```

✓ date pre processing

prompt: give me the cod to show the all unique types of the column suicide date. and show three example of each types. also show the count of each type write tis code as

```

import pandas as pd
def analyze_unique_types(df, column_name, num_examples=3):
    if column_name not in df.columns:
        print(f"Error: Column '{column_name}' not found in the DataFrame.")
        return

    unique_types = df[column_name].apply(type).unique()
    print(f"Unique data types in column '{column_name}':")
    for dtype in unique_types:
        print(f"- {dtype}")

    print(f"\nAnalysis of unique values and their types in column '{column_name}':")
    # Use .apply(type) to get the type of each entry, then value_counts()
    type_counts = df[column_name].apply(type).value_counts()

    for dtype, count in type_counts.items():
        print(f"\nData type: {dtype}")
        print(f"Count: {count}")
        print(f"Examples ({min(num_examples, count)}):")

        # Filter rows by the current data type and get examples
        examples = df[df[column_name].apply(type) == dtype][column_name].head(num_examples).tolist()
        for example in examples:
            print(f"  - {example}")

```

```
analyze_unique_types(df_clr, 'suicide_date')
```

```

Unique data types in column 'suicide_date':
- <class 'str'>
- <class 'datetime.datetime'>
- <class 'float'>

```

```
Analysis of unique values and their types in column 'suicide_date':
```

```

Data type: <class 'str'>
Count: 870

```

```
Examples (3):
- 30/10/20
- 25/9/20
- 24/9/20
```

```
Data type: <class 'datetime.datetime'>
```

```
Count: 714
```

```
Examples (3):
- 2020-04-11 00:00:00
- 2020-08-09 00:00:00
- 2020-12-09 00:00:00
```

```
Data type: <class 'float'>
```

```
Count: 33
```

```
Examples (3):
- nan
- nan
- nan
```

```
import datetime
```

```
def convert_all_to_str(date):
    if isinstance(date, datetime.datetime):
        return date.strftime("%d/%m/%Y")
    elif isinstance(date, str):
        return date # Already a string, leave it as is
    else:
        return None # Covers NaN or any other types
```

```
# Apply the conversion
df_clr['suicide_date'] = df_clr['suicide_date'].apply(convert_all_to_str)
```

```
analyze_unique_types(df_clr, 'suicide_date')
```

```
Unique data types in column 'suicide_date':
```

```
- <class 'str'>
- <class 'NoneType'>
```

```
Analysis of unique values and their types in column 'suicide_date':
```

```
Data type: <class 'str'>
```

```
Count: 1584
```

```
Examples (3):
- 30/10/20
- 11/04/2020
- 25/9/20
```

```
Data type: <class 'NoneType'>
```

```
Count: 33
```

```
Examples (3):
- None
- None
- None
```

```
import re
import pandas as pd
```

```
def analyze_date_format(df, date_column, num_examples=3):
```

```

if date_column not in df.columns:
    print(f"Error: Column '{date_column}' not found in the DataFrame.")
    return

# Initialize counters and example collectors
has_4digit_year = []
no_4digit_year = []
empty_or_null = []

# Loop through the column with index
for idx, val in df[date_column].items():
    if pd.isna(val) or str(val).strip() == "" or val is None:
        empty_or_null.append((idx, val))
    elif re.search(r'\b\d{4}\b', str(val)):
        has_4digit_year.append((idx, val))
    else:
        no_4digit_year.append((idx, val))

# Print summary
print(f"--- Analysis of column '{date_column}' ---")
print(f"✅ Entries with a 4-digit year: {len(has_4digit_year)}")
print(f"Examples ({min(num_examples, len(has_4digit_year))}):")
for idx, val in has_4digit_year[:num_examples]:
    print(f"    - Index {idx}: {val}")

print(f"\n❌ Entries without a 4-digit year: {len(no_4digit_year)}")
print(f"Examples ({min(num_examples, len(no_4digit_year))}):")
for idx, val in no_4digit_year[:num_examples]:
    print(f"    - Index {idx}: {val}")

print(f"\n⚠️ Empty / Null / NaN entries: {len(empty_or_null)}")
print(f"Examples ({min(num_examples, len(empty_or_null))}):")
for idx, val in empty_or_null[:num_examples]:
    print(f"    - Index {idx}: {val}")
print(f"-" * (len(f"--- Analysis of column '{date_column}' ---")))

```

```
analyze_date_format(df_clr, 'suicide_date')
```

```
--- Analysis of column 'suicide_date' ---
```

```
✅ Entries with a 4-digit year: 1450
```

```
Examples (3):
```

```

- Index 1: 11/04/2020
- Index 5: 09/08/2020
- Index 6: 09/12/2020

```

```
❌ Entries without a 4-digit year: 134
```

```
Examples (3):
```

```

- Index 0: 30/10/20
- Index 2: 25/9/20
- Index 3: 24/9/20

```

```
⚠️ Empty / Null / NaN entries: 33
```

```
Examples (3):
```

```

- Index 1292: None
- Index 1301: None
- Index 1317: None

```

```
def fix_two_digit_year(date_str):
    if pd.isna(date_str) or not isinstance(date_str, str) or re.search(r'\b\d{4}\b', date_str):
        return date_str # Already has a 4-digit year or is invalid/null

    parts = date_str.strip().split('/')
    if len(parts) == 3 and len(parts[2]) == 2:
        parts[2] = '20' + parts[2]
        return '/'.join(parts)
    return date_str # Leave untouched if not in expected format
```

```
# Apply to all rows
df_clr['suicide_date'] = df_clr['suicide_date'].apply(fix_two_digit_year)
```

```
analyze_date_format(df_clr, 'suicide_date')
```

```
--- Analysis of column 'suicide_date' ---
```

```
✓ Entries with a 4-digit year: 1584
```

```
Examples (3):
```

- Index 0: 30/10/2020
- Index 1: 11/04/2020
- Index 2: 25/9/2020

```
✗ Entries without a 4-digit year: 0
```

```
Examples (0):
```

```
⚠ Empty / Null / NaN entries: 33
```

```
Examples (3):
```

- Index 1292: None
- Index 1301: None
- Index 1317: None

```
-----
```

▼ categorical cleaing functions

```
def strip_and_lower_columns(df, columns):
    for col in columns:
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip().str.lower().replace('nan', None)
```

```
strip_and_lower_columns(df_clr, categorical_columns)
```

```
def print_unique_values(df, column_name):
    if column_name in df.columns:
        unique_values = df[column_name].unique()
        print(f"Unique values in column '{column_name}':")
        print(unique_values)
    else:
        print(f"Column '{column_name}' not found in the DataFrame.")
```

```
def normalize_column_values(df, mapping_dict, column_name):
    return df[column_name].apply(
```

```
        lambda x: mapping_dict.get(str(x).strip().lower(), x) if pd.notna(x) else None
    )
```

```
print_unique_values(df_clr, 'age_group')
```

```
Unique values in column 'age_group':
['teen' None 'adult' 'middle-aged' 'child' 'youth' 'old' 'baby'
 'middled-aged' 'null' 'youghth' 'young adult' 'young' 'teenager'
 'middle age' 'middle-age']
```

```
age_group_mapping = {
    'baby': 'baby',
    'child': 'child',
    'teen': 'teen', 'teenager': 'teen',
    'young': 'young', 'youghth': 'young', 'young adult': 'young', 'youth': 'young',
    'adult': 'adult',
    'middle-aged': 'middle-aged', 'middle age': 'middle-aged', 'middle-age': 'middle-aged', 'middled-aged': 'middle-aged',
    'old': 'old',
    'null': None,
    None: None
}
```

```
df_clr['age_group'] = normalize_column_values(df_clr, age_group_mapping, 'age_group')
print_unique_values(df_clr, 'age_group')
```

```
Unique values in column 'age_group':
['teen' None 'adult' 'middle-aged' 'child' 'young' 'old' 'baby']
```

```
print_unique_values(df_clr, 'gender')
```

```
Unique values in column 'gender':
['male' 'female' '3rd gender' None 'third gender' 'transgender female'
 'transgender woman']
```

```
gender_mapping = {
    'male': 'male',
    'female': 'female',
    '3rd gender': 'third gender',
    'third gender': 'third gender',
    'transgender female': 'third gender',
    'transgender woman': 'third gender',
    None: None
}
```

```
df_clr['gender'] = normalize_column_values(df_clr, gender_mapping, 'gender')
print_unique_values(df_clr, 'gender')
```

```
Unique values in column 'gender':
['male' 'female' 'third gender' None]
```

```
print_unique_values(df_clr, 'religion')
```

```
Unique values in column 'religion':
['muslim' 'hindu' None 'buddhism' 'buddhist' 'christian' 'chakma' 'marma'
'santal' 'islam']
```

```
religion_mapping = {
    'muslim': 'muslim', 'islam': 'muslim',
    'hindu': 'hindu',
    'buddhism': 'buddhist', 'buddhist': 'buddhist',
    'christian': 'christian',
    'chakma': 'chakma',
    'marma': 'marma',
    'santal': 'santal',
    None: None
}
```

```
df_clr['religion'] = normalize_column_values(df_clr, religion_mapping, 'religion')
print_unique_values(df_clr, 'religion')
```

```
Unique values in column 'religion':
['muslim' 'hindu' None 'buddhist' 'christian' 'chakma' 'marma' 'santal']
```

```
print_unique_values(df_clr, 'marital_status')
```

```
Unique values in column 'marital_status':
[None 'relationship-rejected' 'unmarried' 'relationship' 'married'
'broked_up' 'engaged' 'divorced' 'widowed' 'single' 'broken_up'
'unamrried' 'married (second wife)'
'relationship (indicating a romantic relationship)' 'varied' 'maried'
'unmarired' 'rejected']
```

```
marital_status_mapping = {
    None: None,

    'relationship-rejected': 'rejected',
    'rejected': 'rejected',

    'relationship (indicating a romantic relationship)': 'relationship',
    'relationship': 'relationship',

    'unmarried': 'single',
    'unamrried': 'single',
    'unmarired': 'single',
    'single': 'single',

    'engaged': 'engaged',

    'married': 'married',
    'maried': 'married',
    'married (second wife)': 'married',
    'varied': 'married',

    'divorced': 'divorced',

    'broken_up': 'broken up',
    'broked_up': 'broken up',
```

```
'widowed': 'widowed'
}
```

```
df_clr['marital_status'] = normalize_column_values(df_clr, marital_status_mapping, 'marital_status')
print_unique_values(df_clr, 'marital_status')
```

```
Unique values in column 'marital_status':
[None 'rejected' 'single' 'relationship' 'married' 'broken up' 'engaged'
 'divorced' 'widowed']
```

```
print_unique_values(df_clr, 'profession_group')
```

```
Unique values in column 'profession_group':
['worker' None 'student' 'housewife' 'service holder' 'day labourer'
 'retired' 'business' 'teacher' 'farmer' 'musician' 'saint' 'day laborer'
 'jail chowkidar' 'gambler' 'null' 'actor' 'houswife' 'smuggler' 'doctor'
 'master tailor' 'drivers' 'businessman' 'engineer' 'stportsman'
 'spotsman' 'day-laborer' 'busniess, worker' 'shopkeeper'
 'housewife, student' 'drug dealer' 'driver' 'students' 'police'
 'prisoner' 'politician' 'mental' 'businessmen' 'expatriate' 'expatriates'
 'accused' 'singer' 'in debt' 'an orphan' 'mentally depressed'
 'housekeeper' 'unemployed' 'tourist' 'cultural worker' 'thief'
 'disabled elderly' 'model' 'ngo officer' 'army' 'the widow'
 'the fisherman' 'nurs' 'the prisoner' 'monk' 'asi' 'law enforcement'
 'migrant worker' 'model and actress' 'garment worker' 'police officer'
 'defense' 'fisherman' 'tutor' 'tea seller' 'sales man' 'overseas worker'
 'corporate employee & media professional' 'technician' 'homemaker'
 'military' 'hospitality' 'military/border security' 'artist'
 'educator/principal' 'actress' 'laborer' 'trader' 'medical'
 'rickshaw driver' 'taxi driver' 'handicrafts' 'transportation' 'teaching'
 'student, beautician' 'healthcare' 'former student' 'freelancer'
 'factory worker' 'labour' 'village' 'rickshaw puller' 'employee of dgda'
 'lyricist, former journalist, advertising professional' 'security guard'
 'juvenile detainee' 'nurse' 'hospital staff' 'ice cream seller'
 'building manger' 'auto driver' 'bgb' 'banker' 'pharmacist' 'private job'
 'politics' 'sales representative']
```

```
profession_mapping = {
    # Worker group
    'worker': 'worker', 'labour': 'worker', 'laborer': 'worker', 'day labourer': 'worker',
    'day laborer': 'worker', 'day-laborer': 'worker', 'migrant worker': 'worker', 'garment worker': 'worker',
    'factory worker': 'worker', 'rickshaw puller': 'worker', 'rickshaw driver': 'worker', 'taxi driver': 'worker',
    'auto driver': 'worker', 'tea seller': 'worker', 'ice cream seller': 'worker', 'drivers': 'worker',
    'driver': 'worker', 'busniess, worker': 'worker', 'sales man': 'worker', 'sales representative': 'worker',
    'shopkeeper': 'worker',

    # Student group
    'student': 'student', 'students': 'student', 'former student': 'student', 'student, beautician': 'student',
    'housewife, student': 'student',

    # Housewife / Homemaker group
    'housewife': 'housewife', 'houswife': 'housewife', 'homemaker': 'housewife',

    # Service holder / Corporate / Private job
    'service holder': 'service holder', 'private job': 'service holder',
    'corporate employee & media professional': 'service holder', 'employee of dgda': 'service holder',
```

```
# Retired
'retired': 'retired',

# Business group
'business': 'business', 'businessman': 'business', 'businessmen': 'business',

# Teacher / Educator group
'teacher': 'teacher', 'educator/principal': 'teacher', 'teaching': 'teacher', 'tutor': 'teacher',

# Farmer
'farmer': 'farmer',

# Musician / Artist group
'musician': 'musician', 'singer': 'musician', 'lyricist, former journalist, advertising professional': 'musician',
'artist': 'musician',

# Security / Law enforcement group
'security guard': 'security', 'saint': 'security', 'asi': 'security', 'law enforcement': 'security',
'police': 'security', 'police officer': 'security', 'army': 'security', 'military': 'security',
'military/border security': 'security', 'bgb': 'security', 'defense': 'security',

# Entertainment / Acting group
'actor': 'entertainment', 'actress': 'entertainment', 'model': 'entertainment', 'model and actress': 'entertainment',

# Healthcare group
'doctor': 'healthcare', 'medical': 'healthcare', 'nurse': 'healthcare', 'nurs': 'healthcare',
'hospital staff': 'healthcare', 'pharmacist': 'healthcare', 'healthcare': 'healthcare',

# Criminal-related group
'jail chowkidar': 'criminal', 'prisoner': 'criminal', 'the prisoner': 'criminal', 'accused': 'criminal',
'drug dealer': 'criminal', 'smuggler': 'criminal', 'gambler': 'criminal', 'thief': 'criminal',
'juvenile detainee': 'criminal',

# Politician / Politics
'politician': 'politics', 'politics': 'politics',

# Trader
'trader': 'trader',

# Freelancer
'freelancer': 'freelancer',

# NGO Officer
'ngo officer': 'ngo officer',

# Tourist
'tourist': 'tourist',

# Disabled / Elderly
'disabled elderly': 'disabled elderly',

# Mental health / Vulnerable
'mental': 'mental health', 'mentally depressed': 'mental health', 'in debt': 'mental health', 'an orphan': 'mental health',

# Monk
```

```

'monk': 'monk',

# Village (ambiguous but kept as is)
'village': 'village',

# Building manager (fixed typo from 'building manger')
'building manger': 'building manager',

# Technician
'technician': 'technician',

# Handicrafts
'handicrafts': 'handicrafts',

# Transportation
'transportation': 'transportation',

# Overseas worker
'overseas worker': 'overseas worker',

# Fisherman
'fisherman': 'fisherman', 'the fisherman': 'fisherman',

# Misc / Others
'the widow': 'others',
'null': None,
'master tailor': 'tailor',
'mental health': None,
'expatriates': 'expatriate',
'overseas worker': 'expatriate',
'disabled elderly': 'unemployed',
'handicrafts': 'unemployed',
'village': None,
}

```

```

df_clr['profession_group'] = normalize_column_values(df_clr, profession_mapping, 'profession_group')
print_unique_values(df_clr, 'profession_group')

```

```

Unique values in column 'profession_group':
['worker' None 'student' 'housewife' 'service holder' 'retired' 'business'
'teacher' 'farmer' 'musician' 'security' 'criminal' 'entertainment'
'healthcare' 'tailor' 'engineer' 'stportsman' 'spotsman' 'politician'
'mental health' 'expatriate' 'housekeeper' 'unemployed' 'tourist'
'cultural worker' 'ngo officer' 'others' 'fisherman' 'monk' 'technician'
'hospitality' 'trader' 'transportation' 'freelancer' 'building manager'
'banker']

```

```

print_unique_values(df_clr, 'workplace')

```

```

Unique values in column 'workplace':
['village' None 'urban' 'semi-urban' 'semi-urban' 'buddhist mondir'
'drivingvillage' 'semi-urban (inferred from factory setting)'
'village (kadam rasulpur village)' 'urban (dhaka)' 'rural' 'semi urban'
'village (mohanganj)' 'village (basudebpur village, amarpur union)'
'semi-urban (matiaranga, khagrachari)'
'urban (chittagong college campus)'
'purangor union, uttar boitroni area, satkania' 'sami-urban' 'home-based'

```

```
'dhaka, bangladesh' 'bogura mohammad ali hospita' 'abroad' 'housewife'
'foreign']
```

```
workplace_mapping = {
    None: None,

    # Rural group (includes 'village' and variations)
    'village': 'rural',
    'village (kadam rasulpur village)': 'rural',
    'village (mohanganj)': 'rural',
    'village (basudebpur village, amarpur union)': 'rural',
    'buddhist mondir': 'rural',
    'drivingvillage': 'rural',

    # Urban group (includes 'urban' and variations, cities, abroad)
    'urban': 'urban',
    'urban (dhaka)': 'urban',
    'urban (chittagong college campus)': 'urban',
    'dhaka, bangladesh': 'urban',
    'bogura mohammad ali hospita': 'urban',
    'abroad': 'urban',
    'foreign': 'urban',
    'home-based': 'urban', # assuming home-based is urban

    # Semi-urban group (includes variants and misspellings)
    'semi_urban': 'semi-urban',
    'semi-urban': 'semi-urban',
    'semi urban': 'semi-urban',
    'semi-urban (inferred from factory setting)': 'semi-urban',
    'semi-urban (matiaranga, khagrachari)': 'semi-urban',
    'sami-urban': 'semi-urban',
    'purangor union, uttar boitroni area, satkania': 'semi-urban',
    'housewife': None,

    # Ambiguous/unmapped values could be handled as None or decided later
}
```

```
df_clr['workplace'] = normalize_column_values(df_clr, workplace_mapping, 'workplace')
print_unique_values(df_clr, 'workplace')
```

```
Unique values in column 'workplace':
['rural' None 'urban' 'semi-urban']
```

```
print_unique_values(df_clr, 'financial_condition')
```

```
Unique values in column 'financial_condition':
['indebted' None 'middle' 'lower' 'upper_middle' 'miiddle'
'lower middle class' 'poor' 'rich' 'middle class' 'impoverished/lower'
'middle/lower (implied, family couldn't afford motorcycle)'
'lower-middle or limited financial means (family not affluent, built wicket with own funds)'
'family facing financial hardships, which led to a dispute between fatema and her mother'
'possibly dependent; strained financial support from sons'
'not explicitly stated, but mentioned financial interactions (e.g., tk8,000 requested for travel expenses)'
"extremely poor; unable to afford proper medical treatment due to financial issues and son's lack of income from sales"
'financially weak (family struggled financially)'
"the family was relying on government relief assistance due to the father's illness"
```

```
"not directly stated; however, the family's situation implies they faced difficulties due to bureaucratic issues"
'many belonged to lower-middle-class and low-income families, facing financial difficulties and stress.'
'unemployed during the pandemic, leading to family relationship issues and depression.'
'not directly mentioned, but implied familial stress due to disputes and lack of information on income.'
'job was not mpo (monthly pay order) included, leading to marital discord and mental distress.'
'upper middle' 'middle, indebted' 'upper' 'lower middle' 'middle-class'
'debt' 'likely stable' 'low-income' 'poverty' 'lower_middle']
```

```
financial_condition_mapping = {
    None: None,

    # Indebted / Debt related
    'indebted': 'indebted',
    'middle, indebted': 'indebted',
    'debt': 'indebted',

    # Lower income group (including synonyms and phrases indicating poverty)
    'lower': 'lower',
    'poor': 'lower',
    'impoverished/lower': 'lower',
    'lower middle class': 'lower',
    'lower-middle or limited financial means (family not affluent, built wicket with own funds)': 'lower',
    'financially weak (family struggled financially)': 'lower',
    'extremely poor; unable to afford proper medical treatment due to financial issues and son's lack of income from sales': 'lower',
    'poverty': 'lower',
    'low-income': 'lower',
    'lower middle': 'lower',
    'lower_middle': 'lower',
    'lower_middle': 'lower', # duplicate spelling with underscore
    'many belonged to lower-middle-class and low-income families, facing financial difficulties and stress.': 'lower',
    'family facing financial hardships, which led to a dispute between fatema and her mother': 'lower',
    'possibly dependent; strained financial support from sons': 'lower',
    'the family was relying on government relief assistance due to the father's illness': 'lower',
    'not directly stated, but mentioned financial interactions (e.g., tk8,000 requested for travel expenses)': 'lower',
    'not directly stated; however, the family's situation implies they faced difficulties due to bureaucratic issues': 'lower',
    'unemployed during the pandemic, leading to family relationship issues and depression.': 'lower',
    'not directly mentioned, but implied familial stress due to disputes and lack of information on income.': 'lower',
    'job was not mpo (monthly pay order) included, leading to marital discord and mental distress.': 'lower',

    # Middle income group (including variants and typos)
    'middle': 'middle',
    'middle class': 'middle',
    'middle-class': 'middle',
    'middle': 'middle',
    'middle-class': 'middle',
    'middle/lower (implied, family couldn't afford motorcycle)': 'middle',
    'likely stable': 'middle',
    'upper middle': 'upper_middle',
    'upper_middle': 'upper_middle',

    # Upper income group
    'upper': 'upper',
    'rich': 'upper',
    'not explicitly stated, but mentioned financial interactions (e.g., tk8,000 requested for travel expenses)': None,
}
```

```
df_clr['financial_condition'] = normalize_column_values(df_clr, financial_condition_mapping, 'financial_condition')
print_unique_values(df_clr, 'financial_condition')
```

```
Unique values in column 'financial_condition':
['indebted' None 'middle' 'lower' 'upper_middle' 'upper']
```

```
print_unique_values(df_clr, 'reason')
```

```
Unique values in column 'reason':
['harassment' 'relationship problem' None 'marital affair' 'family issue'
'violence ans mental issue' 'fails' 'humilliation' 'rape' 'poverty'
'violence' 'mental issue' 'fail' 'physical issue' 'voilence'
'relationship problem' 'domestic violence' 'false accusation' 'poisoning'
'money steal accusation' 'addiction'
'family issue or relationship problem' 'null' 'violance'
'domestic violance' 'divorce' 'psysical issue' 'marital affairs'
'depression' 'mental health' 'mentally depressed' 'family feud'
'her husband physical and mental torcher' 'murder his brother'
'he was suffering from various family problems' 'disappointment'
'depressed' 'indebted' 'accused' 'slander' 'in debt' 'an orphan'
'drug addict' 'mentally ill' 'the prisoner' 'in silent sorrow'
'frustration' 'emotional distress' 'societal pressures'
'romantic rejection and emotional distress' 'financial pressure'
'marital discord and emotional distress' 'family dispute with spouse'
'family conflict and possible abuse' 'abuse and humiliation'
'mental distress' 'sexual harassment' 'broken relationships'
'family argument' 'family conflict' 'harassed' 'family issues'
'trauma and depression' 'abandonment and despair' 'mentally unstable'
'social pressure and exposure' 'persistent abuse'
'financial fraud and humiliation' 'sexually abused' 'stress'
'mentally disabled' 'loss in a ludo game' "husband's torture"
'fail the exam' 'disagreement' 'self-motivated' 'failure in love'
'cancer pain' 'mental illness' 'due to pride'
'disappointment over not getting a job' 'violation of personal privacy'
'domestic violence and marital strife' 'blackmailed by a lover'
'marital conflict' 'blackmail, social humiliation'
'psychological effects' 'financial hardship, depression'
'love-induced pride' 'family turmoil' 'family dispute'
'love-induced sadness and emotions' 'pride' 'betrayal in love'
'educational failure' 'marital conflict, mental and physical abuse'
'guilt' 'emotional grief and depression' 'failure' 'family quarrel'
'fail the exam.' 'marital strife and pride' 'unknown' 'domestic conflict'
'personal reasons & emotional distress'
'alleged harassment and abuse by a classmate and an assistant proctor'
"depression, loss of recognition, grief from mother and sister's death"
'protest by students and teachers due to alleged misconduct and irregularities'
'emotional distress from parental divorce and personal struggles'
'psychological issues'
'alleged sexual harassment and related social issues'
'alleged humiliation during arbitration (salis) and subsequent assault'
'conflict with boyfriend, ziauddin rufi, and subsequent emotional distress'
'family issues (family disputes suggested as a reason)'
'depression, emotional struggles'
'relationship issues, emotional distress'
'marital issues, debt, dowry pressure' 'dispute with father'
'mental abuse, police harassment' 'domestic disputes caused by poverty'
'harassment by a local individual' 'public humiliation and assault'
'mental and physical health issues'
'family dispute (argument with parents)'
'emotional distress after husband's suicide'
'emotional distress due to marital disputes'
```

```
'emotional distress due to relationship issues'
'emotional distress and humiliation due to dowry demands'
'chronic illness, financial hardship, inability to afford medicine'
'domestic dispute, unemployment, and financial difficulties'
'loneliness & financial struggles' 'domestic conflict & drug addiction'
```

```
reason_mapping = {
    # harassment
    'harassment': 'harassment', 'sexual harassment': 'harassment', 'harassed': 'harassment',
    'alleged harassment and abuse by a classmate and an assistant proctor': 'harassment',
    'harassment by a local individual': 'harassment', 'harassment by moneylenders': 'harassment',
    'financial distress and harassment from lenders': 'harassment',

    # relationship problems
    'relationship problem': 'relationship', 'relationship problem': 'relationship',
    'relationship issues': 'relationship', 'relationship issue': 'relationship',
    'relationship conflict': 'relationship', 'relationship rejection': 'relationship',
    'relationship issues, emotional distress': 'relationship', 'romantic rejection and emotional distress': 'relationship',
    'broken relationships': 'relationship', 'love relationship conflict': 'relationship',
    'love affair': 'relationship', 'love-induced pride': 'relationship', 'love-induced sadness and emotions': 'relationship',
    'betrayal in love': 'relationship', 'failed relationship': 'relationship', 'failure in love': 'relationship',
    'failure': 'relationship', 'failure to arrange money for going abroad': 'relationship',

    # marital issues
    'marital affair': 'marital', 'marital affairs': 'marital', 'marital discord and emotional distress': 'marital',
    'marital conflict': 'marital', 'marital conflict, mental and physical abuse': 'marital', 'marital distress': 'marital',
    'marital strife and pride': 'marital', 'marital strife': 'marital', 'marital issues, debt, dowry pressure': 'marital',
    'marital betrayal & blackmail': 'marital', 'marriage rejection by in-laws': 'marital',
    'forced marriage & false allegations': 'marital', 'forced marriage dissatisfaction': 'marital',
    'forced marriage pressure': 'marital', 'marital conflict or potential murder (disputed)': 'marital',
    'marital dispute': 'marital', 'domestic violence and marital strife': 'marital', 'domestic violence': 'marital',
    'domestic abuse': 'marital', 'domestic conflict': 'marital', 'domestic disputes caused by poverty': 'marital',
    'domestic dispute': 'marital', 'domestic conflict & drug addiction': 'marital',
    'domestic dispute, unemployment, and financial difficulties': 'marital',
    'family dispute': 'marital', 'family dispute with spouse': 'marital', 'family conflict and possible abuse': 'marital',
    'family feud': 'marital', 'family argument': 'marital', 'family conflict': 'marital', 'family issues': 'marital',
    'family turmoil': 'marital', 'family quarrel': 'marital', 'family related stress': 'marital',
    'family dispute (argument with parents)': 'marital', 'family issues (family disputes suggested as a reason)': 'marital',
    'ongoing family conflicts': 'marital', 'family dispute over not getting dslr camera': 'marital', 'dispute with father': 'marital',
    'argument with mother': 'marital', 'argument with brother': 'marital', 'conflict with mother': 'marital',
    'conflict with boyfriend, ziauddin rufi, and subsequent emotional distress': 'marital',

    # mental health
    'mental issue': 'mental', 'mental distress': 'mental', 'mental and physical health issues': 'mental',
    'mental illness': 'mental', 'mental health': 'mental', 'mental abuse, police harassment': 'mental',
    'mental issues': 'mental', 'mental distress/depression': 'mental', 'mental torture and frustration from the groom over dowry': 'mental',
    'mental health crisis': 'mental', 'mentally depressed': 'mental', 'mentally unstable': 'mental', 'mentally ill': 'mental',
    'mental health condition': 'mental', 'mental distress and humiliation due to dowry demands': 'mental',
    'psychological effects': 'mental', 'psychological issues': 'mental', 'depression': 'mental', 'depressed': 'mental',
    'depression, emotional struggles': 'mental', 'depression, loss of recognition, grief from mother and sister's death': 'mental',
    'depression, social bullying': 'mental', 'likely depression or personal distress': 'mental', 'stress': 'mental',
    'emotional distress': 'mental', 'emotional distress from parental divorce and personal struggles': 'mental',
    'emotional distress and humiliation due to dowry demands': 'mental', 'emotional distress after husband's suicide': 'mental',
    'emotional distress due to marital disputes': 'mental', 'emotional distress due to relationship issues': 'mental',
    'emotional grief and depression': 'mental', 'emotional distress after altercation': 'mental', 'frustration': 'mental',
    'social stigma and humiliation after covid-19 diagnosis': 'mental', 'social humiliation': 'mental', 'in silent sorrow': 'mental',
    'loneliness & financial struggles': 'mental',
```

```

# violence and abuse
'violence': 'violence', 'violence ans mental issue': 'violence', 'voilence': 'violence', 'violance': 'violence',
'domestic violence': 'violence', 'rape': 'violence', 'rape, blackmail, social stigma': 'violence',
'gang rape and social stigma': 'violence', 'blackmail': 'violence', 'blackmail and harassment by former lover (rahim)': 'violence',
'blackmail, social humiliation': 'violence', 'abuse and humiliation': 'violence', 'abuse and torture': 'violence',
'alleged sexual harassment and related social issues': 'violence',
'alleged humiliation during arbitration (salis) and subsequent assault': 'violence', 'public humiliation and assault': 'violence',
'public humiliation': 'violence', 'threat': 'violence', 'false accusation': 'violence', 'false theft accusation': 'violence',
'poisoning': 'violence', 'money steal accusation': 'violence', 'slander': 'violence', 'accused': 'violence',
'suspected foul play': 'violence', 'disputed': 'violence',

# poverty and financial
'poverty': 'financial', 'financial pressure': 'financial', 'financial hardship, depression': 'financial',
'financial fraud and humiliation': 'financial', 'financial exploitation': 'financial', 'financial frustration': 'financial',
'financial issues': 'financial', 'financial issue': 'financial', 'financial stress': 'financial',
'financial stress and domestic conflict': 'financial', 'financial difficulties': 'financial',
'debt pressure': 'financial', 'in debt': 'financial', 'indebted': 'financial', 'debt': 'financial',
'middle, indebted': 'financial', 'likely stable': 'financial',
'not explicitly stated, but mentioned financial interactions (e.g., tk8,000 requested for travel expenses)': 'financial',
'family was relying on government relief assistance due to the father's illness': 'financial',
'job was not mpo (monthly pay order) included, leading to marital discord and mental distress.': 'financial',
'chronic illness, financial hardship, inability to afford medicine': 'financial',

# addiction
'addiction': 'addiction', 'drug addict': 'addiction', 'domestic conflict & drug addiction': 'addiction',

# accidents and health
'accidental discharge': 'accident', 'cancer pain': 'accident', 'health-related mental distress': 'accident',
'medical and financial distress': 'accident', 'throat cancer': 'accident',
'chronic illness, financial hardship, inability to afford medicine': 'accident',
'isolation and mental distress from illness': 'accident', 'death influence': 'accident',

# crime and punishment
'murder his brother': 'crime', 'alleged murder disguised as suicide': 'crime', 'shooting': 'crime', 'suicide': 'crime',

# education failure
'fail': 'education', 'fails': 'education', 'failure': 'education', 'fail the exam': 'education',
'fail the exam.': 'education', 'failed mathematics in ssc exam': 'education', 'ssc exam failure': 'education',
'educational failure': 'education', 'academic exclusion': 'education',

# social pressures and stigma
'societal pressures': 'social', 'social pressure and exposure': 'social',
'social stigma and humiliation after covid-19 diagnosis': 'social', 'social humiliation': 'social',
'bullying/body-shaming': 'social', 'cyberbullying': 'social',

# personal reasons
'personal': 'personal', 'personal conflict and disciplinary restrictions': 'personal',
'personal failure': 'personal', 'personal reasons & emotional distress': 'personal', 'self-motivated': 'personal',
'guilt': 'personal', 'disappointment': 'personal', 'disagreement': 'personal',
'disappointment over not getting a job': 'personal', 'transfer stress and depression': 'personal',
'loneliness & financial struggles': 'personal', 'loss in a ludo game': 'personal',

# unknown
'unknown': None, 'uncertain': None, 'null': None,

```

```
# others
'the prisoner': 'others', 'the widow': 'others', 'an orphan': 'others', 'maternal scolding': 'others',
'lipi had a mental disability': 'others', 'identity fraud by husband, social pressure': 'others',
'online sexual exploitation & family scolding': 'others', 'protest by students and teachers due to alleged misconduct and irregularities': 'others',
'public humiliation': 'others', 'public humiliation and assault': 'others',
'family issue or relationship problem': 'relationship',
'family-related stress': 'family',
'family conflict /': 'family',
}
```

```
df_clr['reason'] = normalize_column_values(df_clr, reason_mapping, 'reason')
print_unique_values(df_clr, 'reason')
```

```
Unique values in column 'reason':
['harassment' 'relationship' None 'marital' 'family issue' 'violence'
'education' 'humiliation' 'financial' 'mental' 'physical issue'
'addiction' 'divorce' 'psysical issue'
'her husband physical and mental torcher' 'crime'
'he was suffering from various family problems' 'personal' 'others'
'social' 'trauma and depression' 'abandonment and despair'
'persistent abuse' 'sexually abused' 'mentally disabled'
"husband's torture" 'accident' 'due to pride'
'violation of personal privacy' 'blackmailed by a lover' 'pride' 'family'
'insult and humiliation by riya's maternal grandmother'
'disagreement over marriage'
'political violence, shock of losing husband' 'child marriage'
'sports-related disappointment' 'economic crisis'
'possibly harassment towards wife' 'institutional harassment'
'suspected family feud' 'grief over wife's sudden death'
'suspected depression' "father's abuse" 'extramarital affair'
"son's elopement against family wishes"
'extramarital affair complications'
'marital conflict with cancer-stricken husband' 'scolding from sister'
'mobile phone dispute'
"failure to get justice for daughter's sexual assault"
'health and poverty' 'parental conflict'
'extramarital relationship fallout']
```

```
reason_mapping_two = {
'family issue or relationship problem': 'family',
'family-related stress': 'family',
'family conflict /': 'family',
'family conflict': 'family',
'psysical issue': 'physical',
'her husband physical and mental torcher': 'physical',
"husband's torture": 'physical',
'humiliation': 'humiliation',
'persistent abuse': 'abuse',
'blackmailed by a lover': 'harassment',
'insult and humiliation by riya's maternal grandmother': 'humiliation',
'political violence, shock of losing husband': 'violence',
'possibly harassment towards wife': 'harassment',
'institutional harassment': 'harassment',
'suspected family feud': 'family',
'suspected depression': 'mental',
"father's abuse": 'physical',
"son's elopement against family wishes": 'family',
```

```

    'extramarital affair complications': 'marital',
    'marital conflict with cancer-stricken husband': 'marital',
    'failure to get justice for daughter's sexual assault': 'harassment',
    'health and poverty': 'financial',
    'parental conflict': 'family',
    'extramarital relationship fallout': 'marital'
}

```

```

df_clr['reason'] = normalize_column_values(df_clr, reason_mapping_two, 'reason')
print_unique_values(df_clr, 'reason')

```

```

Unique values in column 'reason':
['harassment' 'relationship' None 'marital' 'family issue' 'violence'
 'education' 'humiliation' 'financial' 'mental' 'physical issue'
 'addiction' 'divorce' 'physical' 'crime'
 'he was suffering from various family problems' 'personal' 'others'
 'social' 'trauma and depression' 'abandonment and despair' 'abuse'
 'sexually abused' 'mentally disabled' 'accident' 'due to pride'
 'violation of personal privacy' 'pride' 'family'
 'disagreement over marriage' 'child marriage'
 'sports-related disappointment' 'economic crisis'
 'grief over wife's sudden death' 'extramarital affair'
 'scolding from sister' 'mobile phone dispute']

```

```
print_unique_values(df_clr, 'time')
```

```

Unique values in column 'time':
['afternoon' 'night' 'morning' None 'evening' 'noon' 'day' 'moring'
 'niight' 'null' '18:00:00' '21:00:00' '04:30:00' '12:30:00' '17:00:00'
 '17:30:00' '00:30:00' '02:00:00' '22:00:00' '20:30:00' '02:39:00'
 '23:30:00' '01:00:00' '20:00:00' '23:00:00' '09:00:00' '22:30:00'
 '15:00:00' '05:30:00' '18:30:00' '16:30:00' '08:00:00'
 '2025-03-25 14:30:00' '2024-08-29 12:00:00' '2024-05-23 00:00:00'
 '2024-09-07 10:00:00' '2024-04-22 20:00:00' '2024-06-27 11:00:00'
 '20th june 2024' '16th june 2024' '1st july 2024' '31st march 2024'
 '3rd february 2024' '12th august 2024' '20th july 2024'
 '18th may 2024, 8:00 am']

```

```

time_mapping = {
    'afternoon': 'afternoon',
    'morning': 'morning',
    'moring': 'morning',
    'night': 'night',
    'niight': 'night',
    'evening': 'evening',
    'noon': 'noon',
    'day': 'day',
    'null': None,
    'none': None,
    # Specific timestamps mapped manually:
    '18:00:00': 'evening',
    '21:00:00': 'night',
    '04:30:00': 'night',
    '12:30:00': 'afternoon',
    '17:00:00': 'evening',
    '17:30:00': 'evening',

```

```

'00:30:00': 'night',
'02:00:00': 'night',
'22:00:00': 'night',
'20:30:00': 'evening',
'02:39:00': 'night',
'23:30:00': 'night',
'01:00:00': 'night',
'20:00:00': 'evening',
'23:00:00': 'night',
'09:00:00': 'morning',
'22:30:00': 'night',
'15:00:00': 'afternoon',
'05:30:00': 'morning',
'18:30:00': 'evening',
'16:30:00': 'afternoon',
'08:00:00': 'morning',
'2025-03-25 14:30:00': 'afternoon',
'2024-08-29 12:00:00': 'noon',
'2024-05-23 00:00:00': 'night',
'2024-09-07 10:00:00': 'morning',
'2024-04-22 20:00:00': 'evening',
'2024-06-27 11:00:00': 'morning',
'20th june 2024': None,
'16th june 2024': None,
'1st july 2024': None,
'31st march 2024': None,
'3rd february 2024': None,
'12th august 2024': None,
'20th july 2024': None,
'18th may 2024, 8:00 am': 'morning',
}

```

```

df_clr['time'] = normalize_column_values(df_clr, time_mapping, 'time')
print_unique_values(df_clr, 'time')

```

```

Unique values in column 'time':
['afternoon' 'night' 'morning' None 'evening' 'noon' 'day']

```

```

print_unique_values(df_clr, 'method')

```

```

Unique values in column 'method':
[None 'hanging' 'taking pesticide' 'stabbing in the neck' 'taking poison'
'under the wheels of the train' 'jumping from the roof'
'jumping under the train' 'jumping from a moving train'
'taking gas tablet' 'taking toxic tablet' 'cutting his throat'
'jumping in front of a train' 'lying on the railway line'
'taking sleeping pill' 'drinking harpic' 'jumping off the train'
'got poisoned.' 'eaten pesticides' 'falling over train' 'strangulating'
'taking rat killing tablet' 'by pushing injection' 'jumped from rooftop'
'jumped to river' 'cut neck' 'jumping under train' 'eaten tablets'
'taken poison' 'firing himself' 'taken pesticides' 'jumping from roof'
'poison' 'eaten pesticide' 'rat kiling tablets' 'eaten rat kills tablet'
'cutting own neck' 'sleeping pills' 'rat kiling poison'
'cutting neck with bleed' 'kerosine firing herself' 'gas tablet'
'jumping from bridge' 'suicide on railway track'
'by consuming rat poison' 'consumed pesticides' 'poison and hanging'
'by consuming gas tablets' 'used carosine' 'took poison'

```

```
'consumed gas tablets' 'jumping down the train' 'consumed rat poison'
'electric shock' 'by taking poison' 'consumed harpic' 'belt'
'jumped under train' 'jumping from a building'
'by consuming rodenticide (rat poison)' 'jumped from building' 'on fire'
'poisoned' 'gun shoot' 'burning' 'jump form building' 'gas tablets'
'jump form bridge' 'jump under train' 'toilet cleaner'
'cut throat and drink harpic' 'jumping off the building'
'stabbed himself in the stomach' 'jump under water'
'eating sleeping tablets' 'by burning the body' 'cut your own throat'
'poisoning' 'jumped' 'overdose of sleeping pills' 'self-immolation'
'ingestion of pesticide' 'poison ingestion' 'jumping' 'drowning in water'
'pistol shot' 'stabbing self' 'jumped in front of train' 'electrocuted'
'gunshot' 'consumed pesticide' 'jumped from a 6th-floor roof' 'hanged'
'struggling' 'self-inflicted stab wound with a fruit knife'
'jumped in front of a train with daughter' 'hanging with a fan'
'overdose of sleeping pills and other medications' 'suicide' 'train'
'shooting' 'strangulation' 'slitting wrists' 'wrist slitting' 'stabbing'
'cutting throat' 'overdose pills' 'jumping on rail' 'jumping on river'
'jump from rooftop' 'drowning' 'vein cutting' 'suffocation']
```

```
method_mapping = {
    # Hanging & strangulation
    **dict.fromkeys(['hanging', 'hanged', 'hanging with a fan', 'strangulating', 'strangulation', 'struggling', 'belt'], 'hanging'),

    # Overdose & poisoning (all pills, pesticides, poisons, tablets, harpic, kerosine, etc.)
    **dict.fromkeys([
        'taking sleeping pill', 'sleeping pills', 'overdose of sleeping pills', 'overdose of sleeping pills and other medications',
        'eating sleeping tablets', 'overdose pills', 'taking pesticide', 'taking poison', 'poison', 'poisoned', 'poisoning', 'poison ingestion',
        'ingestion of pesticide', 'consumed pesticide', 'consumed pesticides', 'taken poison', 'taken pesticides', 'eaten pesticide',
        'eaten pesticides', 'eaten tablets', 'eaten rat kills tablet', 'rat kiling poison', 'rat kiling tablets', 'taking gas tablet',
        'taking toxic tablet', 'gas tablet', 'gas tablets', 'by consuming gas tablets', 'consumed gas tablets',
        'by consuming rodenticide (rat poison)', 'consumed rat poison', 'by consuming rat poison', 'drinking harpic', 'consumed harpic',
        'cut throat and drink harpic', 'used carosine', 'kerosine firing herself', 'toilet cleaner', 'got poisoned.',
    ], 'overdose'),

    # Jumping
    **dict.fromkeys([
        'jumping from the roof', 'jumping from roof', 'jumped from rooftop', 'jump from rooftop', 'jump form building', 'jump from bridge',
        'jumping from bridge', 'jumping off the building', 'jumping from a building', 'jumped from building', 'jumping off the train',
        'jumping from a moving train', 'jumping in front of a train', 'jumped in front of train', 'jumped in front of a train with daughter',
        'jumping under the train', 'jump under train', 'jumped under train', 'jumping down the train', 'jumping on rail', 'jumping on river',
        'jumping', 'jumped', 'jump form bridge', 'jumping off the train', 'jump from rooftop', 'jump under water',
    ], 'jumping'),

    # Cutting, stabbing, slitting
    **dict.fromkeys([
        'cutting his throat', 'cutting neck with bleed', 'cutting own neck', 'cut throat and drink harpic', 'cut your own throat',
        'cut neck', 'cutting throat', 'stabbed himself in the stomach', 'stabbing self', 'stabbing', 'self-inflicted stab wound with a fruit knife',
        'wrist slitting', 'slitting wrists', 'vein cutting', 'cut throat and drink harpic',
    ], 'cutting'),

    # Shooting/gun
    **dict.fromkeys(['gun shoot', 'gunshot', 'shooting', 'pistol shot'], 'shooting'),

    # Drowning/suffocation
    **dict.fromkeys(['drowning', 'drowning in water'], 'drowning'),
    'suffocation': 'suffocation',
```

```
# Train related accidents
**dict.fromkeys([
    'under the wheels of the train','lying on the railway line','suicide on railway track','train','falling over train',
], 'train accident'),

# Electrocuting & others
'electric shock': 'electrocution',
'electrocuted': 'electrocution',
'by pushing injection': 'injection',
'jumping from bridge': 'jumping',
'jumping from roof': 'jumping',
'jump form building': 'jumping',
'jump form bridge': 'jumping',
'jump under train': 'jumping',
'jump under water': 'drowning',
'jumping under train': 'jumping',
'jumping off the building': 'jumping',
'jumping from a building': 'jumping',
'jumping on rail': 'jumping',
'jumping on river': 'jumping',
'jumping off the train': 'jumping',
'jumping down the train': 'jumping',
}
```

```
df_clr['method'] = normalize_column_values(df_clr, method_mapping, 'method')
print_unique_values(df_clr, 'method')
```

```
Unique values in column 'method':
[None 'hanging' 'overdose' 'stabbing in the neck' 'train accident'
'jumping' 'cutting' 'taking rat killing tablet' 'injection'
'jumped to river' 'firing himself' 'poison and hanging' 'took poison'
'electrocution' 'by taking poison' 'on fire' 'shooting' 'burning'
'drowning' 'by burning the body' 'self-immolation'
'jumped from a 6th-floor roof' 'suicide' 'suffocation']
```

```
method_mapping_two = {
    'stabbing in the neck': 'cutting',
    'taking rat killing tablet': 'overdose',
    'jumped to river': 'jumping',
    'firing himself': 'shooting',
    'poison and hanging': 'hanging',
    'took poison': 'overdose',
    'by taking poison': 'overdose',
    'burning': 'self-immolation',
    'by burning the body': 'self-immolation',
    'on fire': 'self-immolation',
    'jumped from a 6th-floor roof': 'jumping',
}
```

```
df_clr['method'] = normalize_column_values(df_clr, method_mapping_two, 'method')
print_unique_values(df_clr, 'method')
```

```
Unique values in column 'method':
[None 'hanging' 'overdose' 'cutting' 'train accident' 'jumping']
```

```
'injection' 'shooting' 'electrocution' 'self-immolation' 'drowning'  
'suicide' 'suffocation']
```

```
print_unique_values(df_clr, 'weather_main')
```

```
Unique values in column 'weather_main':  
['haze' 'clouds' 'drizzle' 'rain' 'clear' 'mist' 'thunderstorm' None  
 'partly cloudy']
```

```
print_unique_values(df_clr, 'weather_description')
```

```
Unique values in column 'weather_description':  
['haze' 'scattered clouds' 'drizzle' 'moderate rain' 'overcast clouds'  
 'broken clouds' 'clear sky' 'light rain' 'mist' 'few clouds'  
 'heavy intensity rain' 'thunderstorm with rain' 'thunderstorm'  
 'thunderstorm with light rain' None 'cloudy']
```

```
print_unique_values(df_clr, 'hometown')
```

[Show hidden output](#)

```
hometwon_mapping = {  
    'gopalgang': 'gopalganj',  
    'barisal': 'barishal',  
    'habiganj': 'habiganj',  
    'manikganj': 'manikganj',  
    'narayangang': 'narayanganj',  
    'kustia': 'kushtia',  
    'sirajganj': 'sirajganj',  
    'bogra': 'bogura',  
    'comilla': 'cumilla',  
    'chattogram': 'chattogram',  
    'munshiganj': 'munshiganj',  
    'naogaon': 'naogaon',  
    'lalmonirhat': 'lalmonirhat',  
    'kisharganj': 'kishorganj',  
    'norshinde': 'narsingdi',  
    'nouga': 'naogaon',  
    'jessore': 'jashore',  
    'potuakhali': 'patuakhali',  
    'borguna': 'barguna',  
    'bagherhat': 'bagerhat',  
    'jaypurhat': 'joypurhat',  
    'shitakundo': 'sitakunda',  
    'natore baraigram': 'natore',  
    'dinajpur bochaganj': 'dinajpur',  
    'rangpur pirgachha': 'rangpur',  
    'sherpur,nakla': 'sherpur',  
    'chadpur,kachua': 'chandpur',  
    'shirajganj': 'sirajganj',  
    'chadpur': 'chandpur',  
    'borolouho ghar, akhaura, maniyondo, brahmanbari': 'brahmanbaria',  
    'muradnagar,comilla': 'cumilla',  
    'shyamoli, dhaka': 'dhaka',  
    'rupnagar, dhaka': 'dhaka',
```

```

'nur chala area of bhatara, dhaka': 'dhaka',
'baruipara, godagari, rajshahi': 'rajshahi',
'jotragnab, bagha, rajshahi': 'rajshahi',
'binodpur union of mohammadpur upazila, magura': 'magura',
'kamargao, mograpara union, sonargaon, narayanganj': 'narayanganj',
'jammura, kalikapur, choudhogram, comilla': 'cumilla',
'maheshpur, shorisha union, ishwarganj, mymensingh': 'mymensingh',
'noyapara railway station, madhupur, habiganj': 'habiganj',
'kalainchura, baniachang, habiganj': 'habiganj',
'gabchia, indurkandi, pirojpur': 'pirojpur',
'bhandaria, pirojpur': 'pirojpur',
'choyghoria village, boikari union, satkhira': 'satkhira',
'south pathorghata, jhaudanga, satkhira': 'satkhira',
'boromchal, kulaura': 'moulvibazar',
'fulbaria, mymensingh': 'mymensingh',
'raninagar, naogaon': 'naogaon',
'putimari, shingra, nator': 'natore',
'motlob, chadpur': 'chandpur',
'hajiganj, chandpur': 'chandpur',
'vayna, joshore': 'jashore',
'kedarpur, keshobpur, joshore': 'jashore',
'tatipara, nilgonj, joshore': 'jashore',
'srinagar, patuakhali': 'patuakhali',
'dumki, patuakhali': 'patuakhali',
'lokkhipur, dosmina, patuakhali': 'patuakhali',
'kaichail, nagorkanda, faridpur': 'faridpur',
'sohosrail, sekhon, boalmari, faridpur': 'faridpur',
'alipur, azmiriganj, habiganj': 'habiganj',
'birashi, lehamba union, ranishongkail, thakurgaon': 'thakurgaon',
'birashi bazar area of ranisankail upazila, thakurgaon': 'thakurgaon',
'goreya chongakhata, thakurgaon': 'thakurgaon',
'chorsugona, ghosherpara, melandoho, jamalpur': 'jamalpur',
'shreenagar, munshiganj': 'munshiganj',
'gandhacchi, bangodda, nangolkot, comilla': 'cumilla',
'dastapur, jaypurhat': 'joypurhat',
'sonarban bashbari, amarthan, panchagarh': 'panchagarh',
'chorkhukshia, dhunot, bogra': 'bogura',
'patikpara, hatibandha, lalmonirhat': 'lalmonirhat',
'tinua village of haripur upazila, thakurgaon': 'thakurgaon',
'noyanagor, gojaria union, munshiganj': 'munshiganj',
'jindabazar, sylhet': 'sylhet',
'puthia village of ullahpara upazila, sirajgonj': 'sirajgonj',
'horinchora union, nilphamari': 'nilphamari',
'munshinangla, melandoho, jamalpur': 'jamalpur',
'munsefpara, brahmanbaria': 'brahmanbaria',
'amirabad union, feni': 'feni',
'narayanpur, gosinga, shreepur, gazipur': 'gazipur',
'barura upazila, chittagong': 'chattogram',
'shingur, barura, comilla': 'cumilla',
'mokimabad, hajiganj, chandpur': 'chandpur',
'silimpur, tajhat, rangpur': 'rangpur',
'barhatta, netrokona': 'netrokona',
'milpara, basudebpur, amarpur union, chiribondor, dinajpur': 'dinajpur',
'piera, gona, raninagar, naogaon': 'naogaon',
'abdar, telihati, shreepur, gazipur': 'gazipur',
'chokropara, matiranga, khagrachori': 'khagrachari',

```

```
'chattagram': 'chattogram',  
'bir kamotthali, nondail, mymensingh': 'mymensingh',  
'chandipur, sujatpur, baniachang, habigonj': 'habiganj',  
'taifachora, belchori, matiranga, khagrachori': 'khagrachari',  
'mondutosh, vangura, pabna': 'pabna',  
'boropara, south hashimpur, chandnaish, chattogram': 'chattogram',  
'kolatoli, monpura bhola': 'bhola',  
'chunarughat, habigonj': 'habiganj',  
'lakhai, hobigonj': 'habiganj',  
'kamarjuri, gazipur': 'gazipur',  
'bolakipur, baniachang, habigonj': 'habiganj',  
'jirat, narendrapur, joshore': 'jashore',  
'botna village of gosairhat upazila, shariatpur': 'shariatpur',  
'bhangura upazila of pabna': 'pabna',  
'chelopara, bogra': 'bogura',  
'kochuakhali, lalmohon, vola': 'bhola',  
'tejkunipara, tejgaon, dhaka': 'dhaka',  
'jhalkathi, barishal': 'jhalokati',  
'ashtolapara, jibonnagar, chuadanga': 'chuadanga',  
'thakurpur, damurhuda, chuadanga': 'chuadanga',  
'palgram, rauthgaw, kulaura, moulovibazar': 'moulvibazar',  
'hasupur, handial union, chatmohor, pabna': 'pabna',  
'dhaowapara, bogra': 'bogura',  
'naruli south dhawapara, bogra': 'bogura',  
'kumarkhali upazila, kushtia': 'kushtia',  
'araihazar upazila, narayanganj': 'narayanganj',  
'moulvipara, gopalganj': 'gopalganj',  
'pachani bari, rajar kha, south up, shohorasti, chandpur': 'chandpur',  
'goalondo, rajbari': 'rajbari',  
'lakhai upazila, habiganj': 'habiganj',  
'begunai, lakhai, habigonj': 'habiganj',  
'auliapukur, chiribondor, dinajpur': 'dinajpur',  
'bokshigonj, jamalpur': 'jamalpur',  
'maheshpur, jhinaidaha': 'jhenaidaha',  
'west bilashpur, gazipur': 'gazipur',  
'ibrahimpur, damurhuda, chuadanga': 'chuadanga',  
'fulbari, dinajpur': 'dinajpur',  
'doulotdia, goalondo, rajbari': 'rajbari',  
'gopsono village in keshabpur upazila, jessore': 'jashore',  
'bakoshpota, maheshpur, jhinaidoho': 'jhenaidaha',  
'bojrapur, maheshpur, jhinaidaha': 'jhenaidaha',  
'inderjani, kakrajan, sakhipur, tangail': 'tangail',  
'shahebganj, rupsha, faridgonj, chandpur': 'chandpur',  
'madhabpur, habigonj': 'habiganj',  
'bodnikhali, bura mojumder, betagi, borguna': 'barguna',  
'kosba, brahmanbaria': 'brahmanbaria',  
'lalpur, brahmonbaria': 'brahmanbaria',  
'bakshipur pal, shoilkupa, jhinaidaha': 'jhenaidaha',  
'jhikragacha, magura': 'magura',  
'tiorpara, santahar, adamdighi, bogra': 'bogura',  
'saluapara, sajiara, magura': 'magura',  
'dakatia, joshore': 'jashore',  
'sowdapara, dewalbari, melandoho, jamalpur': 'jamalpur',  
'badalgachi, naogaon': 'naogaon',  
'gopsena, keshobpur': 'jashore',  
'badeghonapara, gosaipur, shreebordi, sherpur': 'sherpur',
```

```
'mirgonj, raypur, lokkhipur': 'lakshmipur',  
"hindu para, palongkhali, ukhia, cox's bazar": 'coxsbazar',  
'bhanga upazila': 'faridpur',  
'shabdakar, shreenathpur, alinagar, moulovibazar': 'moulvibazar',  
'companygonj, noakhali': 'noakhali',  
'kesmot, hoibotpur, joshore': 'jashore',  
'moniharpara, sapahar, naogaon': 'naogaon',  
'bandortila, epz, chattogram': 'chattogram',  
'rohomot para, bashbaria union, shitakundo': 'sitakunda',  
'falsi, harinakundu, jhinaidaha': 'jhenaidah',  
'miajibari, baropaika, rupsa, faridgonj, chandpur': 'chandpur',  
'panchagar railway, dinajpur': 'panchagarh',  
'rahimpur, muradnagar, comilla': 'cumilla',  
'sokhipur, tangail': 'tangail',  
'deyapara, abhaynagar, joshore': 'jashore',  
'noagao, lakhai, habigonj': 'habiganj',  
'polashbari, gaibandha': 'gaibandha',  
'guakhola, abhaynagar, jessore': 'jashore',  
'junaki, shalikha, magura': 'magura',  
'hakimpur, dinajpur': 'dinajpur',  
'kuti union, brahmanbaria': 'brahmanbaria',  
'moricha, boiragichor, doulatpur, kushtia': 'kushtia',  
'west chatni, adamdighi, bogra': 'bogura',  
'paikosha, jhaoil, kamarkhand, sirajgonj': 'sirajganj',  
'ujanishar, brahmanbaria': 'brahmanbaria',  
'asondipara, shreebordi, mirpur': 'sherpur',  
'bashbaria, vatagram, nandigram, bogra': 'bogura',  
'khetlal, jaypurhat': 'joypurhat',  
'raypur, lakshipur': 'lakshmipur',  
'mohipur, jamtola, bogra': 'bogura',  
'boalkhali upazila, chattogram': 'chattogram',  
'dattapara, tangi': 'gazipur',  
'binodpur, rajbari': 'rajbari',  
'hariharnagar of manirampur, jessore': 'jashore',  
'west jurain, shampur, dhaka': 'dhaka',  
'balutungi, choudala, chapainobabgonj': 'chapainawabganj',  
'darus salam, dhaka': 'dhaka',  
'shingra upazila, natore': 'natore',  
'momarijpur, dagonvua, feni': 'feni',  
'habibnagar area of college road, rangpur': 'rangpur',  
'sodupur, atraï, naogaon': 'naogaon',  
'baful, patuakhali': 'patuakhali',  
'thanapara, kushtia': 'kushtia',  
'buraburi, ulipur, kurigram': 'kurigram',  
'bahadurpur, veramara, kushtia': 'kushtia',  
'ramgoti, lokkhipur': 'lakshmipur',  
'komolnagar, lakshipur': 'lakshmipur',  
'kafrul, mirpur, dhaka': 'dhaka',  
'modonpur, kaligonj, lalmonirhat': 'lalmonirhat',  
'basudebpur, harirampur, manikgonj': 'manikganj',  
'bhatara, satura, manikgonj': 'manikganj',  
'paruabil, komolgonj, moulovibazar': 'moulvibazar',  
'halimpur, moulovibazar': 'moulvibazar',  
'islamabad, kulaura, moulovibazar': 'moulovibazar',  
'sirajdikhan, munshigonj': 'munshiganj',  
'kharha, shreenagar, munshigonj': 'munshiganj',
```

```
'kamaldi, araihazar, narayangonj': 'narayanganj',  
'alokbali, norshingdi': 'narsingdi',  
'borai, natore': 'natore',  
'polshura, kandipara, natore': 'natore',  
'gobindoshree, modon, netrokona': 'netrokona',  
'chorhazari, companygonj, noakhali': 'noakhali',  
'lahuria, lohagora, narail': 'narail',  
'amkhola, golachipa, patuakhali': 'patuakhali',  
'charokgachia, mothbaria, pirojpur': 'pirojpur',  
'gulisakhali, mothbaria, pirojpur': 'pirojpur',  
'mondalpara, goalando, rajbari': 'rajbari',  
'bhatshala, debhata, shatkhir': 'satkhira',  
'hazarhari, khalilnagar, tala, satkhira': 'satkhira',  
'jessor': 'jessore',  
'vhola': 'bhola',  
'narsinggdi': 'narsingdi',  
'cox's bazar': 'coxsbazar',  
'nehari para, sylhet': 'sylhet',  
'rauzan, chattogram': 'chattogram',  
'barkal, chandanaish, chattogram': 'chattogram',  
'bahadurpur village, singra thana, natore district': 'natore',  
'char jabbar police complex, noakhali': 'noakhali',  
'charkalkini union, lakshmipur': 'lakshmipur',  
'goalanda, ward no. 9, alam chowdhury para': 'rajbari',  
'narsingdi district': 'narsingdi',  
'bangalipur, syedpur': 'syedpur',  
'sujonkathi village, agailjhara upazila, barishal': 'barishal',  
'hamidi village, vanga thana, faridpur': 'faridpur',  
'hathazari, chittagong': 'chattogram',  
'khalisha baherduli, chandpur': 'chandpur',  
'sonatala, bogura district': 'bogura',  
'ujirpur, barishal district': 'barishal',  
'sandwip, bauria, chattogram': 'chattogram',  
'bishwakolony area, akbar shah, chattogram': 'chattogram',  
'rangunia, chittagong': 'chattogram',  
'gajaria, munshiganj': 'munshiganj',  
'nandigram, kushtia': 'kushtia',  
'jamalganj, sunamganj': 'sunamganj',  
'ullapara, sirajganj': 'sirajganj',  
'kalirhat, aditmari, rangpur': 'rangpur',  
'sariakandi upazila, bogra': 'bogra',  
'chittagong, bandar thana': 'chattogram',  
'ramchandradi village, gopaldi municipality, araihazar upazila': 'narayanganj',  
'srinagar, munshiganj': 'munshiganj',  
'reserve bazar, rangamati': 'rangamati',  
'mujrai, sripur': 'gazipur',  
'elisha, bhola sadar': 'bhola',  
'ataikula pal para, raninagar, naogaon': 'naogaon',  
'tamta south union, shahrasti, chandpur': 'chandpur',  
'padua munshi mura, lohagara, chittagong': 'chattogram',  
'char afzal, ramgati, lakshmipur': 'lakshmipur',  
'gopinathpur, gurdaspur, natore': 'natore',  
'tepirbari, sreepur, gazipur': 'gazipur',  
'cheunia, kaliganj, jhenaidah': 'jhenaidah',  
'jheena kumoriapara, raninagar, naogaon': 'naogaon',  
'swagaon village, lakhai, habiganj': 'habiganj',
```

```

'garkanda village, nalitabari, sherpur': 'sherpur',
'patuakhali, mirzaganj upazila': 'patuakhali',
'keraniganj, kathpatti riverbank': 'dhaka',
'sreepur, khubjipur, gurdaspur, natore': 'natore',
'ballamjhar, gaibandha': 'gaibandha',
'rangabali, patuakhali': 'patuakhali',
'east shilertua, rupasipara union, lama, bandarban': 'bandarban',
'charkocha, lalmohan, bhola': 'bhola',
'shyamnagar, satkhira': 'satkhira',
'sarail, brahmanbaria': 'brahmanbaria',
'ukhiya no. 7 rohingya camp, cox's bazar': 'coxsbazar',
'ramakantpur village, singair, manikganj': 'manikganj',
'belkuchi, sirajganj': 'sirajganj',
'matlabattar, chandpur': 'chandpur',
'boalkhali municipality, chittagong': 'chattogram',
'phulpur, mymensingh': 'mymensingh',
'chaudagram, comilla': 'comilla',
'epz, chittagong': 'chattogram',
'haripur, kushtia': 'kushtia',
'amki village, sonaimuri, noakhali': 'noakhali',
'chandgaon village, daudkandi, cumilla': 'comilla',
'bakal village, agoiljhara upazila, barishal district': 'barishal',
'bandarban, ward no. 4, madhyampara, near votoghata': 'bandarban',
'cumilla sadar, bangladesh': 'comilla',
'gopalganj (paternal), lalmonirhat (husband's residence)': 'gopalganj',
'sonamukhi village, mehendiganj, barishal': 'barishal',
'cumilla, bangladesh': 'comilla',
'uttar boitroni, purangor union, satkania, chattogram': 'chattogram',
'charagaon, shrimantagao, tahirpur, sunamganj': 'sunamganj',
'south bhattakhawa village, dewanganj, jamalpur': 'jamalpur',
'sharifbag, dharmarai, dhaka': 'dhaka',
'bishgiripara, nalitabari, sherpur': 'sherpur',
'dabua union, raouzan, chattogram': 'chattogram',
'lal khan bazar, tankir pahar, chattogram': 'chattogram',
'kolapara, patuakhali (dhanakhali union)': 'patuakhali',
'haridrabadia village, gulishakhali union, amtali upazila, barguna': 'barguna',
'char lawrence union, ward 4, kamalnagar upazila, lakshmipur': 'lakshmipur',
'গোকুল্ডা ইউনিয়ন, গুড়িয়াদহ গ্রাম, লালমনিরহাট সদর': 'lalmonirhat',
'কুল্দী, লোহাগড়া পৌরসভা, নড়াইল': 'narail',
'সোনাইমুখী, মেহেন্দিগঞ্জ, বরিশাল': 'barishal',
'ছাতিয়ানতলা গ্রাম, দামুড়হুদা উপজেলা, চুয়াডাঙ্গা': 'chuadanga',
'স্বরুসোনা গ্রাম, শালিখা উপজেলা, মাগুরা': 'magura',
'সরিষাবাড়ী উপজেলা, জামালপুর': 'jamalpur',
'হাইদগাঁও ইউনিয়ন, পটিয়া, চট্টগ্রাম': 'chattogram',
'হাসানপুর গ্রাম, পীরগঞ্জ থানা, রংপুর': 'rangpur',
'কুমিল্লা, বাংলাদেশ': 'comilla',
'borohoibatpur, sadar upazila, jessore': 'jessore',
'bokimpur, tongibari, munshiganj': 'munshiganj',
'tanore municipality, rajshahi': 'rajshahi',
'haridrabadia, amtali, bangladesh': 'barguna',
'bakhiganj, jamalpur, bangladesh': 'jamalpur',
'haidgaon union, patiya, chittagong, bangladesh': 'chattogram',
'sathkania, chattogram': 'chattogram',
'sahapur union, azizlata area': 'unknown',
'balotra, rajasthan': 'india',
'sharifpur, jessore': 'jessore',

```

```
"rayerbagh, dhaka": "dhaka",
"bagatipara upazila, natore": "natore",
"tungibaria village, rangabali upazila, patuakhali": "patuakhali",
"murshidabad": "india",
"adarshapara, jhenaidah sadar upazila": "jhenaidah",
"kendua, netrokona": "netrokona",
"shilkhali chakma para, cox's bazar": "cox's bazar",
"bararia village, magura district": "magura",
"raghob area, rajarhat, kurigram": "kurigram",
"baia purba para, shahzadpur, sirajganj": "sirajganj",
"nagaljhara, satkhira": "satkhira",
"palestine": "other",
"rupganj, narayanganj": "narayanganj",
"thanapara, gaibandha": "gaibandha",
"bakal, barishal": "barishal",
"usa": "other",
"rajman village, ullapara, sirajganj": "sirajganj",
"syedpur choto begunbari, thakurgaon": "thakurgaon",
"sylhet, akhalia": "sylhet",
"gomastapur, chapainawabganj": "chapainawabganj",
"rajshahi, sheikhpura area": "rajshahi",
"ranishankail upazila, thakurgaon": "thakurgaon",
"baliadangi upazila, thakurgaon": "thakurgaon",
"bashbari village, puthia, rajshahi": "rajshahi",
"north chakmarkul, ramu upazila, cox's bazar": "cox's bazar",
"cumilla": "comilla",
"ilishpur village, kalaroa, satkhira": "satkhira",
"kurmaikul village, rangunia, chattogram": "chattogram",
"pathandanga dulabari village, gaibandha sadar upazila": "gaibandha",
"jabarhat karnai village, pirganj upazila": "thakurgaon",
"magura district": "magura",
"sumirdia village, chuadanga": "chuadanga",
"ahmedpur village, hajiganj upazila, chandpur": "chandpur",
"old dhaka": "dhaka",
"pashcimtara village, thakurgaon": "thakurgaon",
"gangenandapur village, jhikargacha, jashore": "jessore",
"kushtia, bangladesh": "kushtia",
"jowarkorer hat, chattogram": "chattogram",
"banshkhali, chattogram": "chattogram",
"sujaipur, jashore": "jessore",
"sirajganj, bangladesh": "sirajganj",
"balakandi village, sokhipur upazila, shariatpur": "shariatpur",
"kaharole upazila, dinajpur": "dinajpur",
"nageshwari upazila, kurigram": "kurigram",
"paikgachha upazila, khulna": "khulna",
"islampur union, eidgaon upazila, cox's bazar": "cox's bazar",
"ramanander aak village, rajihar union, agoiljhara": "barishal",
"sultan colony, double mooring, chattogram": "chattogram",
"pangkha area, subarnachar, noakhali": "noakhali",
"madhya bharaterchara, bhurungamari, kurigram": "kurigram",
"pingna west para, sarishabari, jamalpur": "jamalpur",
"shukkur bariarhat, dhoom union, mirsharai": "chattogram",
"morelganj, bagerhat": "bagerhat",
"belgharia eastpara, yusufpur, charchat, rajshahi": "rajshahi",
"jamtala area, meradia, khilgaon, dhaka": "dhaka",
"babupur, samsherganj": "murshidabad",
```

```

"doani moniram village, kandi union, pargachha upazila, rangpur": "rangpur",
"sapna building, chankharpul, dhaka": "dhaka",
"bagichartek, rampura, dhaka": "dhaka",
"south 24 parganas": "india",
"sonaipur, ramgarh, khagrachhari": "khagrachhari",
"baigartek, pallabi, dhaka": "dhaka",
"manikchhari, khagrachhari": "khagrachhari",
"daopara hindu para, sherpur, bogura": "bogura",
"adarsha para, jhenaidah sadar": "jhenaidah",
"chilaura-haldipur union, jagannathpur, sunamganj": "sunamganj",
"tupiwala para area, chattogram": "chattogram",
"syedpur upazila, nilphamari": "nilphamari",
"pailgaon union, jagannathpur, sunamganj": "sunamganj",
"goaler kanda, pathakata union-6, nakla, sherpur": "sherpur",
"chuadanga sadar": "chuadanga",
"nazigati konagram, madhiati, gopalganj sadar": "gopalganj",
"singerbari, madhupur, tangail": "tangail",
"chilahat, nilphamari": "nilphamari",
"baralidaha, sreepur, magura": "magura",
"piplubaria, daulatpur, kushtia": "kushtia",
"north mojuipur, lakshmipur": "lakshmipur",
"sombhupur union, tajumuddin, bhola": "bhola",
"lohanipara union, badarganj, rangpur": "rangpur",
"pacchor union, shibchar, madaripur": "madaripur",
"jigatola, hazaribagh, dhaka": "dhaka",
"feniakul area, khagrachhari": "khagrachhari",
"sadar union, durgapur, netrokona": "netrokona",
"shyamalagachi uttarpara, sharsha, jashore": "jessore",
"keshabpur, jashore": "jessore",
"ghurka bazar, salanga, sirajganj": "sirajganj",
"math salta, faridpur": "faridpur",
"bidhar, singra, natore": "natore",
"shantinagar, rupganj": "narayanganj",
"shahagola, atrai, naogaon": "naogaon",
"raja palang, ukhiya, cox's bazar": "cox's bazar",
"enayet bazar, chattogram": "chattogram",
"hajipara, mongla": "bagerhat",
"paschim bachhpur, juri, moulvibazar": "moulvibazar",
"mastul balur math, khilket": "dhaka",
"hajipara, mongla, bagerhat": "bagerhat",
"uttar langaljhara, kalaroa, satkhira": "satkhira",
"digha, raona, gafargaon, mymensingh": "mymensingh",
"gafargaon, mymensingh": "mymensingh",
"badamtala, bally, howrah": "india",
"gilabari, saptibari, aditmari, lalmonirhat": "lalmonirhat",
"akbar shah, chattogram": "chattogram",
"dombarichala, gazipur": "gazipur",
"sundolpur, daudkandi, cumilla": "comilla",
"kholiajani, mirzapur, tangail": "tangail",
"charabagh, nagarpur, tangail": "tangail",
"jamalkhan, chattogram": "chattogram",
"solakura, belkuchi, sirajganj": "sirajganj",
"nimaichara, chatmohar, pabna": "pabna",
"chandpur bridge": "chandpur",
"sitakunda, chattogram": "chattogram",
"seral, agailjhara": "barishal",

```

```

"betaga, south tripura": "india",
"par gendaria, south keraniganj, dhaka": "dhaka",
"east madarbari, chattogram": "chattogram",
"nayekpur, madan, netrakona district": "netrokona",
"hossenpur, nilgonj union, kalapara, patuakhali": "patuakhali",
"naomala, bauphal, patuakhali": "patuakhali",
"barokote village, borobari union, baliadangi, thakurgaon": "thakurgaon",
"rajiyar union, agoiljhara, barisal": "barishal",
"birajala, madarbunia union, sadar, patuakhali": "patuakhali",
"meradia, khilgaon, dhaka": "dhaka",
"ward-1, joyag union, sonaimuri, noakhali": "noakhali",
"betagi, barguna": "barguna",
"alinagar union, kamalganj upazila, moulvibazar": "moulvibazar",
"mahilara union, gournadi, barisal": "barishal",
"amilaish, satkania, chattogram": "chattogram",
"garghori village, alamdanga, chuadanga": "chuadanga",
"khagrachori": "khagrachhari",
"mirakathi, sadar upazila, bikrampur": "munshiganj",
"bikromkols village, munshibazar union, moulvibazar": "moulvibazar",
"south alinagar (akbar nagar), mirsarai, chattogram": "chattogram",
"momin nagar naodaga, bogura": "bogura",
"siddhirganj": "narayanganj",
"sharankhola, bagerhat": "bagerhat",
"basabo, dhaka": "dhaka",
"jayrampur village, damurhuda, chuadanga": "chuadanga",
"bagachra, sharsha, jashore": "jessore",
"bogadana union, sonagazi, feni": "feni",
"kasra union, jessore sadar upazila": "jessore",
"goutampara, suhilpur union, brahmanbaria": "brahmanbaria",
"rajakhali union, pekua, cox's bazar": "cox's bazar",
"alisharkul village, sreemangal, moulvibazar": "moulvibazar",
"khan jahan ali thana, khulna": "khulna",
"daulatkhalibazarpara village, doulatpur upazila, kushtia": "kushtia",
"ratia rajapur, dhansagar, shoronkhola, bagerhat": "bagerhat",
}

```

```

df_clr['hometown'] = normalize_column_values(df_clr, hometwon_mapping, 'hometown')
print_unique_values(df_clr, 'hometown')

```

```

Unique values in column 'hometown':
['brahmanbaria' 'sylhet' 'manikganj' 'bogura' 'dhaka' 'gopalganj' 'pabna'
'chapainawabgang' 'kurigram' 'natore' 'satkhira' 'barishal' 'rajshahi'
'netrakona' 'bankura' 'jamalpur' 'narayanganj' 'mymensingh' 'madhabpur'
'sunamganj' 'nilphamari' 'khulna' 'gazipur' 'narsingdi' 'madaripur'
'rajbari' 'chandpur' 'kishorganj' 'barguna' 'bagerhat' 'jhalokati'
'habiganj' 'sherpur' 'mawa' 'chapainawabganj' 'sirajganj' 'jhenaidah'
'jaipurhat' 'noakhali' 'feni' 'pirojpur' 'patuakhali' 'panchagarh'
'shariatpur' 'bhola' 'tangail' 'kushtia' 'cumilla' 'chattogram'
'bandarban' 'gaibandha' 'magura' 'thakurgaon' 'munshiganj' 'sunamgang'
'dinajpur' 'jessore' 'munshiganj' 'naogaon' 'rangpur' 'khagrachari'
'narail' 'chittagong' 'rangamati' 'shibganj' 'netrokona' 'lalmonirhat'
'moulvibazar' 'laxmipuri' 'jashore' 'fulbari'
'quechuatail village chandrima rajshahi' 'nator bagatipara'
'sarishabari' 'faridpur' 'pirganj, rangpur' 'sayedpur' 'meherpur'
'sitakunda' 'sonargaon' 'nabinagar' 'joypurhat' 'amtola village, narail'
'maddhobasudebpur, hakimpur, dinajpur' 'pach kalia, doulotpur'

```

```
'madhupur, tangail' 'vuiyabari, hornidurgapur, faridgonj, chandpur'
'haripur, thakurgaon' 'solla, noakhali'
'jethua village, tala upazila, satkhira' 'hospital road, barishal'
'baniachor, jalilpar, muksudpur, gopalganj'
'koinpura, anowara, chattogram' 'dhakuria, monirampur'
'noralia, barobkundo union, shitakundo'
'tetia kandapara, bishnondi, araihazar, narayanganj'
'shingur, barura, comilla' 'chudadanga' 'lakshmipur' 'coxsbazar'
'joshore' 'hinda, gangni' 'gazipur, shreepur' 'moulovibazar'
'durgapur, netrokona' 'jhalakathi' 'kishoreganj' 'syedpur' 'keraniganj'
'bhatalia' None 'gournadi, barishal'
'(singuria, anehla union, ghatail, tangail' 'jhalokathi' 'dhamrai, dhaka'
'mirpur, dhaka' 'hapania, gournadi, barishal'
'ghilato1 , jaintapur, sylhet' 'rajapur upazila, jhalokati'
'gangnagar majhpara, shibganj, bogura' 'chalitabaria, sharsha, jessore'
'mohonpur, rajshahi' 'banani, dhaka' 'mohammadpur, sheikhertak' 'bogra'
'beradi village, satair union' 'sughat, sherpur' 'payra kunj, mirzaganj'
'bagharpara, jessore' 'jockey nagar, rajanagar' 'puthia, rajshahi'
'lama, bandarban' 'kaliganj' 'azimpur, dhaka'
'brahmanbaria, akhaura, mogra, chaigharia' 'haridhali village' 'bhairav'
'nagarkanda, faridpur' 'comilla' 'shyampur, dhaka' 'donia, dhaka'
'mugda, dhaka' 'uttara west, dhaka' 'barishal, bangladesh'
'gokul village, tanore municipality, rajshahi' 'unknown' 'india'
'cox's bazar' 'other' 'murshidabad' 'khagrachhari']
```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename = '1_cleaned.xlsx'

# # Save the DataFrame to an Excel file
# df_clr.to_excel(filename, index=False)

# # Download the file
# files.download(filename)
```

bottom